马晨熙

大语言模型推理系统 / 机器学习系统工程师

北京 | +86 15387087330 | shannon20051127@gmail.com | github.com/algebra-MCX

教育背景

武汉理工大学 2024 年 9 月 - 2028 年 6 月

机械工程·本科 武汉，中国

开源工程经历

vLLM Project 2026 年 10 月 – 至今

开源贡献者·*Model Runner V2 / DBO / MoE* *GitHub*

对于 vLLM MRV2，DBO 尤其适合 MoE 模型、Expert Parallelism（EP）、高并发 Decode，以及通信开销较大的分布式推理场景。

一、MRV2 DBO 执行正确性与异常处理

- **专家路由索引修复（已合并）：** #63372（Merged）。修复 MRV2 DBO 在 Microbatch 执行过程中 Routed Expert 数据写入的索引错误，根据 Microbatch 的 Token Offset 正确定位专家路由结果。在 32 请求的 DeepSeek-V2-Lite 测试中，路由一致率由主分支的 0% 提升至 89.4%。
- **Microbatch 异常传播（已合并）：** #63476（Merged）。改进 `UBatchRunner.finish()` 的异常处理逻辑，报告最先记录的 Microbatch 异常，避免原始错误被后续异常覆盖。新增 H200 GPU 回归测试，2/2 测试通过。
- **DBO Worker 故障恢复（Draft）：** #64718（Draft）。针对某个 Microbatch 执行失败后其他 Worker 阻塞于同步等待的问题，设计异常发布与等待者唤醒机制，防止 `UBatchRunner.finish()` 永久挂起，并保留原始异常。该 PR 计划在 MRV1 依赖移除工作 #64668（Draft）完成后继续集成。

二、CUDA Graph 与推理性能优化

- **MLA Decode Staging（Open）：** #63556（Open）。扩展 MRV2 的 MLA Decode Staging，支持不均衡 Data Parallel 负载下捕获的 FULL Microbatch CUDA Graph，优化 DBO 场景中的 Graph Replay 执行。方案已进入 Maintainer 技术讨论，目前正在评估替代实现。
- **性能验证：** 在 63/97 负载 A/B 实验中，输出吞吐量提升 15.52%，p95 Time Per Output Token（TPOT）从 132.017 ms 降至 113.930 ms，约降低 13.7%。

三、MoE 通信正确性：NIXL / DeepEP

- **NIXL Expert Mapping（Open）：** #64086（Open）。修复 Global Expert ID 到 Physical Expert ID 映射过程中无效专家 ID 的处理问题，保留 `-1` Padding Sentinel，避免无效专家被错误映射为合法索引。已获得 Reviewer 对代码修改的积极反馈，等待后续 Ready / CI 流程。
- **DeepEP LL Expert Mapping（Open）：** #64340（Open）。修复 DeepEP Low-Latency Dispatch / Combine 路径中 Round-Robin Expert Remapping 的 Sentinel 传播问题，统一映射逻辑与回归测试。已获得 Reviewer 对代码修改的积极反馈，等待后续 Ready / CI 流程。
- **正确性验证：** 覆盖混合有效/无效专家 ID、全 Padding 及线性映射场景；NIXL 与 DeepEP LL 两种后端共 6 项 CPU Adapter 回归测试全部通过。

四、MRV2 DBO 架构重构与 MRV1 依赖移除

- **核心执行路径解耦(Draft)：** #64668(Draft)。重构 MRV2 DBO 的 Microbatch Slicing 机制,移除对 MRV1 `maybe_create_ubatch_slices` 的依赖，在 MRV2 `create_ubatch_slices` 中直接计算 Padded Slices，保持 Microbatch 边界与 Padding 语义一致，推进 MRV2 DBO 执行路径独立化。消除跨 Runner 的冗余切片计算，为后续彻底移除 MRV1 遗留实现提供支持。简化 Microbatch 切分的数据流，减少对旧版 Runner 内部实现细节的依赖。
- **测试解耦与架构优化：** 移除 MRV2 测试对 MRV1 `split_attn_metadata` 的依赖，为 5 种 Batch Layout 固定预期切分结果；独立迁移 `UBatchWrapper` Padding Mask 测试，降低跨 Runner 耦合，提升模块可维护性与回归测试独立性。通过显式测试 Oracle 固定切分行为，降低后续 DBO 重构引入行为回归的风险。为 MRV2 后续独立演进及 DBO 功能扩展建立更清晰的模块边界与测试基础。

实习经历

Meshy AI 2026 年 4 月–2026 年 8 月

推理系统工程实习生

- **编译与算子优化：**为 Venus Image-to-3D 双阶段 Flow 模型接入 Regional `torch.compile`，针对动态顶点数量启用 `dynamic=True`，利用 Triton Kernel Fusion 减少 Elementwise 开销。Stage 1 去噪循环的 GPU 利用率从 37% 提升至 65%。
- **CFG 与去噪优化：**将 Conditional / Unconditional CFG 双次 Forward 合并为 Batch=2 的单次 Forward，减少约 50% 的 Kernel Launch；通过预计算与复用 Image / Voxel Context、RoPE Cache 和 Attention Metadata 等循环不变量，减少重复计算，GPU 利用率最高达到 95%。
- **Mesh Decoder 优化：**按顶点度数分组批量执行 Sinkhorn，合并 GPU-to-CPU 数据传输，消除逐顶点同步瓶颈；使用 CPU 多线程池并行执行 Linear Assignment Problem（LAP），将 6,250 顶点场景的 Mesh Decoder 延迟从约 20s 降至不足 1s。
- **端到端性能：**在保持生成结果一致的前提下，将 600 顶点场景的推理延迟从 8.7s 降至 5.2s（降低约 40%）；6,250 顶点场景从 26.8s 降至 10.4s （约 2.6× 加速）。

研究经历

华盛顿大学系统实验室（UW Systems Lab） 2026 年 5 月 – 至今

研究实习生（*Research Intern*） 美国西雅图 / 远程

指导教授： Baris Kasikci 和 Arvind Krishnamurthy

- 作为第三作者参与论文 *TraceLab: Characterizing Coding Agent Workloads for LLM Serving*（arXiv:2606.30560），研究 Coding Agent 工作负载的执行模式及其对 LLM Serving 系统的影响。
- 参与分析约 4,300 个 Coding Agent 会话、350,000 次 LLM 推理步骤和 430,000 次工具调用，研究请求分布、上下文长度和工具交互行为。
- 分析长时间自主执行循环、长输入与短输出、工具调用重尾分布及 Prefix Cache 命中率，为 Agent 场景下的 LLM Serving 性能建模与系统优化提供实证依据。

清华大学 2025 年 11 月–2026 年 3 月

研究实习生（*Research Intern*） 中国北京 / 远程

- **Transformer 推理计算图：**基于 C++ / GGML 为 JittorInfer 实现 Qwen2.5 推理计算图，涵盖 RMSNorm、QKV Projection、Grouped-Query Attention（GQA）、RoPE、KV Cache、SwiGLU FFN 和 LM Head，完成模型架构分发及计算图构建流程的集成。
- **Ascend NPU 与 Graph Engine：**实现 Qwen2.5 的普通计算图与 GE 专用执行路径，扩展 Attention Indices、Query / KV Length Metadata、Attention Mask 等输入处理，对接 GGML KV Cache 与 GE Attention 计算，支持昇腾 NPU 上的图构建和执行路径适配。
- **推理框架集成：**扩展 Qwen2.5 模型参数解析、Tokenizer 与 GGUF 加载支持，集成 CANN Flash Attention 相关 Mask 处理，并补充模型配置、CMake 构建和 Ascend NPU 推理启动脚本。

上海交通大学 2025 年 8 月–2025 年 11 月

研究实习生（*Research Intern*） 中国上海 / 远程

- **请求调度与连续批处理：**研究 LLM Serving 中的 Continuous Batching、Prefill / Decode 调度与请求抢占机制，分析不同并发负载下的 GPU 利用率、TTFT、TPOT 及吞吐量表现。
- **KV Cache 内存管理：**研究 Paged KV Cache、Block Allocation、Prefix Caching 和 Cache Eviction 策略，设计不同上下文长度与请求到达分布下的缓存管理实验。
- **性能建模与基准测试：**构建不同 Batch Size、输入输出长度及并发度下的推理性能测试，分析调度开销、KV Cache 容量与推理尾延迟之间的关系。这里主要是我正式入门大模型推理系统的经历，所以也列出来了。

技术技能

推理系统： vLLM、LLM Serving、MRV2、Dual-Batch Overlap、Microbatch Execution、CUDA Graphs、Prefill / Decode

分布式与 MoE： Data Parallelism（DP）、Expert Parallelism（EP）、NIXL、DeepEP、Expert Routing、Dispatch / Combine

性能与正确性：GPU Profiling、Benchmarking、Throughput / TPOT、CUDA Graph Replay、Regression Testing、Failure Propagation

开发工具： Python、PyTorch、Triton、torch.compile、pytest、Git / GitHub、Linux、NVIDIA H200 / B200